

Production Backend Design: HRMS with Task Module

Executive Summary

We propose a **Java Spring Boot + PostgreSQL** backend for an SMB HRMS/Payroll SaaS with integrated task assignment. Key features include HR (employees, attendance, leave), Payroll (salary, taxes, payslips), and a robust Task module (tasks, subtasks, templates, time logs, approvals, payroll integration, notifications, attachments, audit trails, roles). We recommend a modular architecture (initially monolith with clear service layers or microservices) containerized via Docker/K8s for scalability. APIs will be RESTful with JSON (detailed endpoints for tasks and payroll), backed by a relational schema (normalized tables with tenant isolation, indexing, partitioning for large data).

We provide complete specifications: feature list, high-level architecture diagrams, module breakdown (Auth, User, HR, Payroll, Task, TimeTracking, Notifications, FileStore, Audit, Reporting), event flows, and deployment topology. The report includes full API specs (JSON schemas, auth scopes, error codes, filters), database DDL (tables, columns, keys, indexes, sample data), ER and sequence diagrams (via Mermaid), implementation patterns (Spring packages, entities, repos, services, transactions, concurrency controls, schedulers, messaging), data access optimization (GIN indexes, Redis caching, pagination), file storage (S3 presigned URLs, scanning, retention), notification design, security/RBAC/compliance, multi-tenancy (shared schema + RLS ¹), testing, observability, and DevOps (Docker, Helm, CI/CD, DB migrations).

An explicit AI prompt is included at the end to generate the full Spring Boot project (entities, controllers, services, tests, CI configs) from this spec. This document is structured as a developer-ready specification, with tables, JSON examples, diagrams, and inline notes on unspecified details.

1. Features (MVP + Advanced)

HR Module (core): Employee CRUD, departments, designations, roles/permissions, attendance (punch, shift scheduling), leave management, company policies.

Payroll Module: Salary structures, allowances/deductions, tax calculations (TDS, PF/ESI, PT), payslip generation, statutory reports (Form 16, tax filings), payroll runs (monthly, off-cycle). Support multiple pay frequencies and custom bonus/arrears.

Task Module (core): Task creation/assignment with title/desc, due dates, status (Pending/In Progress/Done), priorities. Allow **subtasks**. Task **templates** (e.g. onboarding checklist). Time log entry per task (user, date, hours). **Approval workflow:** tasks can require manager approval. **Task→Payroll links:** mark tasks as “billable” or “bonus-worthy”, so approved hours/bonus feed into payroll.

Notifications: Automated alerts via Email/SMS/WhatsApp/Push when tasks are assigned/updated/overdue, attendance reminders, payroll approvals. Configurable templates.

Attachments & Comments: File attachment to tasks (documents/images). Comment threads on tasks. Audit trail logs all actions (create/update/approve) with timestamps and user info.

Roles/Permissions: RBAC with roles (Admin, HR Manager, Manager, Employee). Field-level visibility. OAuth2/JWT auth with scopes (read:tasks, write:payroll, etc).

Integrations: Email (SMTP/SendGrid), SMS (Twilio/Msg91), WhatsApp Business API, Push (FCM), Third-party AD/LDAP (optional), accounting/ERP (CSV export), attendance devices, calendar (Google).

Advanced: Recurring tasks, task dependencies, templates catalog, analytics (task completion rates), full-text search (Postgres GIN), multi-tenant tenancy management.

2. System Architecture

High-level Diagram: A containerized backend exposing REST APIs, with modules for Auth, Users, HR, Payroll, Task, TimeTracking, Notifications, FileStore, Audit, Reporting. Optionally separate microservices (e.g. Notification service, File service). External DB (Postgres) and caches (Redis).

【Architecture Diagram: A Spring Boot service cluster behind API gateway, connected to Postgres, Redis, S3, and notification providers.】

Service Boundaries: Initially a modular monolith is simplest. Modules (packages) can later be split: - **Auth Service:** OAuth2/JWT. - **User/Directory Service:** Tenant, User, Role, Permission. - **HR Service:** Employee profile, attendance, leave. - **Task Service:** Task, Subtask, Template, TimeLog, Approval. - **Payroll Service:** Salary config, runs, payslips. - **Notification Service:** Sends emails/SMS/WhatsApp. - **File Service:** Manages S3 uploads. - **Audit/Logging Service:** Captures all audit logs. - **Reporting/Analytics Service:** Task reports, payroll metrics.

Event Flows: Use event-driven (Kafka/RabbitMQ). E.g. TaskCompleted event → triggers time allocation; PayrollApproved event → notify user.

Deployment Topology: Docker containers orchestrated with Kubernetes (Helm charts). CI/CD pipelines (GitHub Actions/Jenkins) build images and apply manifests. Each service has readiness probes, horizontal pod autoscaling. DB with read replicas (if needed). Load balancer (Ingress).

3. API Specification

All endpoints use HTTPS and require a Bearer JWT. JSON for request/response.

Auth & Users

- `POST /api/auth/login` - body: {username, password}, resp: {token, expires}.

- `POST /api/auth/refresh` – refresh token flow.
- `GET /api/users?tenant={id}` – list users (perm: ADMIN). Paging, filter by role.
- `POST /api/users` – create user (body: {username, email, roleIds, ...}).
- `GET /api/users/{id}`, `PUT /api/users/{id}`, `DELETE /api/users/{id}`.

HR Endpoints (example)

- `GET /api/employees` – list employees.
- `POST /api/employees` – create employee.
- `GET /api/employees/{id}`, `PUT`, `DELETE`.
- `POST /api/attendance/punch` – record clock-in/out.

(Similar CRUD for leave, departments, etc.)

Task Endpoints

- `GET /api/tasks` – list tasks. Query params: `assigneeId`, `status`, `dueBefore`, `completed`, pagination (`page`, `size`), sort (by `dueDate`).
- `GET /api/tasks/{taskId}` – get task details.
- `POST /api/tasks` – create task (body: {title, description, assigneeId, dueDate, priority, templateId?}).
- `PUT /api/tasks/{taskId}` – update (status, title, etc).
- `DELETE /api/tasks/{taskId}` – delete (Admin).
- `POST /api/tasks/{taskId}/subtasks` – create subtask (parentTaskId is implied).
- `GET /api/tasks/{taskId}/subtasks` – list subtasks.
- `POST /api/tasks/{taskId}/comments` – add comment (body: {text}).
- `GET /api/tasks/{taskId}/attachments` – list attachments.
- `POST /api/tasks/{taskId}/attachments` – upload (multipart form-data).
- `POST /api/tasks/{taskId}/logtime` – log time (body: {userId, date, hours}).
- `GET /api/tasks/{taskId}/timelogs` – list time logs.
- `POST /api/tasks/{taskId}/approve` – manager approves completion (body: {approvedBy, bonusAmount?}).

Auth Scopes: E.g. `role:task:write`, `role:payroll:read`.

Errors: Standard HTTP codes. 400/422 for validation, 401 unauthorized, 403 forbidden, 404 not found, 500 server error. Error JSON: `{code:"INVALID_INPUT", message:"...", field:"..."}`.

Example: Create Task

```

POST /api/tasks
Headers: Authorization: Bearer <token>
Body:
{
  "title": "Monthly PF Filing",
  "description": "Prepare and file PF returns",
  "assigneeId": 15,
  "dueDate": "2026-04-25",
  "priority": "High"
}
Response 201:
{
  "id": 101,
  "title": "Monthly PF Filing",
  "status": "Pending",
  "dueDate": "2026-04-25",
  "assignee": {"id":15,"name":"Priya"}
}

```

4. Database Design (PostgreSQL)

Schema Highlights: Tenant isolation via `tenant_id`. Shared schema with RLS recommended ¹. Use UUIDs for keys. Timestamp with time zone for date fields. Audit tables for data changes.

Tables (DDL-style):

```

CREATE TABLE tenants (
  id UUID PRIMARY KEY,
  name TEXT UNIQUE NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE users (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  username TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  role_id UUID NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(tenant_id, username)
);

```

```
CREATE TABLE roles (
  id UUID PRIMARY KEY,
  name TEXT NOT NULL,
  permissions JSONB NOT NULL -- e.g. ["PAYROLL_RUN","TASK_APPROVE"]
);
```

```
CREATE TABLE employees (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  user_id UUID REFERENCES users(id),
  name TEXT NOT NULL,
  department TEXT,
  designation TEXT,
  email TEXT,
  phone TEXT,
  join_date DATE,
  salary BIGINT, -- monthly salary in cents
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE tasks (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  title TEXT NOT NULL,
  description TEXT,
  status TEXT NOT NULL DEFAULT 'Pending', -- Pending, InProgress, Completed
  priority TEXT DEFAULT 'Normal', -- Low, Normal, High
  assignee_id UUID REFERENCES users(id),
  created_by UUID REFERENCES users(id),
  due_date DATE,
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMP,
  bonus_amount BIGINT DEFAULT 0, -- in cents
  completed_at TIMESTAMP,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE subtasks (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  parent_task_id UUID NOT NULL REFERENCES tasks(id),
  title TEXT NOT NULL,
  status TEXT DEFAULT 'Pending',
```

```

    assignee_id UUID REFERENCES users(id),
    due_date DATE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE time_logs (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    task_id UUID NOT NULL REFERENCES tasks(id),
    user_id UUID NOT NULL REFERENCES users(id),
    log_date DATE NOT NULL,
    hours NUMERIC(5,2) NOT NULL, -- e.g. 8.50
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE attachments (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    task_id UUID REFERENCES tasks(id),
    filename TEXT NOT NULL,
    file_path TEXT NOT NULL, -- S3 key or URL
    uploaded_by UUID REFERENCES users(id),
    uploaded_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    entity TEXT NOT NULL, -- e.g. 'tasks'
    entity_id UUID NOT NULL,
    action TEXT NOT NULL, -- CREATE, UPDATE, DELETE
    changes JSONB, -- diff of changes
    performed_by UUID,
    performed_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE payroll_runs (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    period_start DATE NOT NULL,
    period_end DATE NOT NULL,
    executed_at TIMESTAMP NOT NULL DEFAULT NOW(),

```

```
    executed_by UUID REFERENCES users(id)
);
```

```
CREATE TABLE payslips (
  id UUID PRIMARY KEY,
  payroll_run_id UUID NOT NULL REFERENCES payroll_runs(id),
  employee_id UUID NOT NULL REFERENCES employees(id),
  base_salary BIGINT NOT NULL,
  overtime_amount BIGINT DEFAULT 0,
  bonus_amount BIGINT DEFAULT 0,
  deductions JSONB, -- e.g. {TDS:5000,PF:3000}
  net_pay BIGINT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

Indexes: - CREATE INDEX ON tasks (tenant_id, assignee_id, status, due_date); - GIN index on JSONB (permissions, deductions). - CREATE INDEX ON time_logs (task_id); - PARTITION large tables by time (e.g. time_logs monthly, payroll_runs yearly). - RLS: enforce tenant_id = current_setting('app.tenant_id').

Partition/Archival: TimeLogs partitioned by year/month, old partitions archived to cheaper storage. Audit logs partitioned by year.

Sample Rows (JSON):

Tenants:

```
{"id":"T001","name":"Acme Corp","created_at":"2026-01-01T00:00:00Z"}
```

Tasks:

```
{"id":"101","tenant_id":"T001","title":"File TDS","status":"Pending","priority":"High","assignee_id":"U15","created_by":"U1","due_date":"2026-
```

TimeLogs:

```
{"id":"501","task_id":"101","user_id":"U15","log_date":"2026-04-24","hours":2.5}
```

Payslips:

```
{
  "id": "PS999",
  "payroll_run_id": "PR202604",
  "employee_id": "E42",
  "base_salary": 50000,
  "overtime_amount": 7500,
  "bonus_amount": 2000,
  "deductions": {
    "TDS": 6000
  },
  "net_pay": 54000
}
```

5. Diagrams

ER Diagram (Mermaid): Entities (Tenant, User, Role, Employee, Task, Subtask, TimeLog, Attachment, PayrollRun, Payslip). Relationships: Task→Assignee(User), Subtask→ParentTask, TimeLog→Task/User, Payslip→PayrollRun/Employee.

```
erDiagram
    TENANTS ||--o{ USERS : contains
    USERS ||--o{ EMPLOYEES : linked
    ROLES ||--o{ USERS : assigned
    USERS ||--o{ TASKS : assignedTasks
    TASKS ||--|{ SUBTASKS : has
    TASKS ||--o{ TIME_LOGS : logs
    USERS ||--o{ TIME_LOGS : logs
    TASKS ||--o{ ATTACHMENTS : attaches
    PAYROLL_RUNS ||--o{ PAYSLIPS : generates
    EMPLOYEES ||--o{ PAYSLIPS : has
```

Sequence: Task Completion & Payroll Integration (Mermaid):

```
sequenceDiagram
    participant M as Manager
    participant E as Employee
    participant S as System
    participant P as Payroll
    M->>S: Create Task & assign to E
    S->>E: Notify via Email/SMS
    E->>S: Log time (hours)
    E->>S: Mark Task Completed
    S->>M: Notify completion pending approval
    M->>S: Approve Task
    S->>Audit: Log action
    S->>P: Trigger payroll entry (hours/bonus)
```

Sequence: User Onboarding:

```
sequenceDiagram
    participant A as Admin
```



```
participant S as System
A->>S: Create Tenant + Admin user
S->>Auth: Generate OAuth credentials
A->>S: Add Company Info
A->>S: Add Employees, Assign Roles
S->>A: Send welcome email with setup links
```

6. Backend Implementation

- **Project Structure:**

- Use Maven/Gradle.
- Packages: `com.company.auth`, `user`, `hr`, `payroll`, `task`, `timelog`, `notification`, `filestore`, `audit`, `reporting`, `config`.
- Each has Controllers, Services, Repositories, DTOs, Entities.

- **Key Classes:**

- Entities annotated with JPA (`@Entity`).
- Repositories extend `JpaRepository` (with custom queries for complex filters).
- Services handle business logic (transactional methods).
- Controllers map REST endpoints.
- **DTOs & Mappers:** DTO classes for requests/responses. Use MapStruct or manual mapping from Entities to DTOs.
- **Transactions:** Use `@Transactional` on service methods (e.g. task creation logs + notifications). Propagation: default REQUIRED.
- **Optimistic Locking:** Add `@Version` field (integer) on entities like Task, Payslip to detect concurrent updates.
- **Concurrency Control:** Use database constraints for uniqueness. For critical flows (e.g. payroll run), use `@Transactional` and locks if needed (SELECT FOR UPDATE).
- **Background Jobs:** Use Spring's `@Scheduled` or Quartz for recurring tasks (e.g. daily reminder checks, monthly payroll schedule).
- **Event-Driven:** Use Kafka or RabbitMQ. E.g. on TaskApproved event, publish a message consumed by PayrollService to recalc payroll. Use Spring Cloud Stream or RabbitTemplate.

7. Data Access & Performance

- **Queries:** Use pagination (`Pageable`) for list endpoints. Example: `Page<Task> findByAssigneeIdAndStatus(UUID assignee, String status, Pageable page)`.

- **Indexing:** Index `tasks(tenant_id, status)`, `time_logs(task_id)`, `payslips(payload_run_id)`. Use GIN index on JSONB (e.g. deductions) for query.
- **Full-Text Search:** If needed, add `tsvector` column on `tasks(title, description)` and GIN index for keyword search.
- **Caching:** Use Redis for frequently-read but rarely-changed data (e.g. department list, static config). Use Spring Cache abstraction.
- **Bulk Operations:** Support bulk create (e.g. uploading time logs) via CSV import. Use batch inserts or `JdbcTemplate.batchUpdate` for efficiency.
- **Lazy vs Eager:** Carefully use LAZY loading on relations to avoid N+1.

8. File Storage & Attachments

- **S3 Integration:** Store attachments in S3 bucket. Backend generates **pre-signed URL** for upload (PUT).
- **Virus Scanning:** (Design note) Upon S3 upload event, a Lambda or container can scan files. Mark attachments as quarantined until clean.
- **Retention:** Delete old attachments after tenant-specific retention policy (e.g. 7 years for compliance) via scheduled job.
- **Metadata:** Store path, file size, content-type in DB. No binary in DB.

9. Notifications Design

- **Providers:** Email via JavaMail or services (SendGrid). SMS via Twilio/MSG91 (India). WhatsApp Business API. Push via Firebase.
- **Templates:** Use templating (Thymeleaf or simple string templates) with placeholders for user/task fields.
- **Retry/Rate-limit:** Use a message queue (RabbitMQ) to enqueue notifications. Implement retry on failure (e.g. 3 attempts). Throttle SMS/WhatsApp by provider limits (~1req/sec).

10. Security & Compliance

- **Auth:** OAuth2 with JWT (Spring Security OAuth2). Tokens carry tenant ID and role claims.
- **RBAC:** Roles/permissions enforced in code (`@PreAuthorize`). Data-level: always filter by `tenant_id`. Use PostgreSQL RLS for multi-tenant data isolation ¹.
- **Encryption:** Encrypt sensitive fields (e.g. SSN, if any) with AES and key management. Use TLS for all transport.
- **Audit Trails:** All DB changes also written to `audit_logs` table. Sensitive actions (salary change) require two-person audit.
- **Consent:** If storing biometrics or health info (unspecified fields), capture consent timestamp.
- **Privacy Law:** Adhere to India's PDP Act (2023). Expose data deletion on request, minimize PII storage.
- **Logging & Monitoring:** Structured logs (JSON) with trace IDs. Include security logs (login attempts).

- **Breach Response:** In event of breach, notify affected tenants per regulation.

11. Multi-Tenancy

- **Approach:** Shared database & schema with a `tenant_id` column on all tables is simplest (Model 1)
¹ . Use Postgres RLS for strict isolation.
- **Onboarding:** New tenant creation workflow (generate UUID, seed default roles).
- **Billing Hooks:** Flag per tenant feature toggles (e.g. Task module on/off). Charge by number of employees or fixed subscription.
- **Tenant Isolation:** Enforce `tenant_id` in all queries (client middleware sets `app.current_tenant`).
- **Scaling:** If client demands, migrate heavy tenants to separate schema or DB (Model 2/3).

12. Testing & CI

- **Unit Tests:** JUnit/Mockito for services and utilities.
- **Integration Tests:** Spring Boot tests with `@DataJpaTest` for repos, `@SpringBootTest` for flows (use `Testcontainers` for Postgres).
- **Contract Tests:** If services talk via HTTP, use Pact or similar.
- **Load Testing:** Simulate 1000 tasks/hour (Gatling or JMeter).
- **CI Pipeline:** On push, run static analysis (SpotBugs), tests, build Docker image, publish to registry.

13. Observability

- **Metrics:** Expose Prometheus metrics (via Micrometer) for request latencies, DB query times, scheduler job success, queue depths.
- **Tracing:** Integrate OpenTelemetry for distributed tracing (trace Task API → DB → notifications).
- **Logging:** Centralize logs (ELK stack or CloudWatch). Log request IDs, user IDs, tenant IDs.
- **Alerts:** Set alerts on error rates, high latency, CPU/memory usage.

14. DevOps & Deployment

- **Docker:** Write Dockerfile for Spring Boot JAR. Multi-stage build (compile, then run).
- **Helm/K8s:** Helm chart with Deployment, Service, ConfigMap (secrets from Vault).
- **DB Migration:** Use Flyway or Liquibase for schema changes. Migrations stored in VCS.
- **Backup/DR:** Automated nightly DB backups (AWS snapshots). Replication for HA. Test restore periodically.
- **CI/CD:** Pipeline to build/test and `kubectl apply`. Use blue/green or canary for zero-downtime.
- **Secrets:** Use K8s Secrets or Vault for DB creds, API keys.

15. Scalability & Cost

- For ~100 tenants (small scale), one DB instance and 2-3 app replicas suffice. For ~10,000 tenants, use Read Replicas, partitioning, and scale horizontally.
- Estimate: 4 vCPU, 16GB RAM for Postgres primary; each app node 2 vCPU/4GB. Autoscale by CPU or queue lag.
- K8s cluster with 3 master, 5 worker nodes (4 CPU, 16GB each) can support mid-size usage.

- Cloud costs: DB (RDS), storage (S3), networking. Scale out as tenants grow.

16. Migration & Versioning

- **DB Migration:** Flyway scripts ensure reproducible schema updates. Use semantic versioning for migrations.
- **API Versioning:** Prefix endpoints with `/api/v1/`. For breaking changes, create `/v2/`. Support backward compatibility during rollout.

17. Developer Documentation & AI Prompt

Detailed README will include: architecture overview, setup steps, module descriptions, and developer guides. Below is an AI code-generation prompt:

AI Prompt (Code Generation):

"Generate a complete Spring Boot application for a multi-tenant HRMS/Payroll backend with a Task module as specified:

- Entities (with JPA) and tables: Tenant, User, Role, Employee, Task, Subtask, TimeLog, Attachment, PayrollRun, Payslip, AuditLog, etc. (see schema above). Include relationships and fields.
- Repositories (Spring Data) and Services for each module.
- REST Controllers for all endpoints (Auth, Users, Employees, Tasks, TimeLogs, Payroll) with request/response DTOs (use JSON examples from spec). Implement pagination, filtering.
- Use Spring Security OAuth2 with JWT and RBAC (roles/permissions).
- Include background jobs (Spring Scheduler) for reminders and payroll runs.
- Integrate with PostgreSQL. Use optimistic locking on Task/Payslip (`@Version`).
- Include sample unit and integration tests (with Testcontainers for Postgres).
- Provide application.yml for multi-tenant configuration (e.g. RLS).
- Include Dockerfile, docker-compose or K8s manifest, and Flyway migration scripts to create schema.
- Generate OpenAPI (Swagger) documentation and a Postman collection for all APIs.
- Structure code under packages: auth, user, hr, payroll, task, notification, audit, filestore, etc.
- Provide sample seed data inserts in SQL.
- Follow the earlier specification for endpoints, DB tables, fields, and sample JSON."

This prompt guides an AI to scaffold the entire backend project with correct entities, APIs, and configuration.

18. Sample Data & API Docs

- **SQL Seed Data:** Provide INSERT statements for a sample tenant, admin user, one employee, one task, one payroll run.
- **Postman Collection:** Export JSON file with requests for major flows (authentication, create task, log time, run payroll).
- **OpenAPI (Swagger):** Annotations in code or separate YAML spec covering all endpoints with schemas and examples.

19. Roadmap (1/3/6/12 months)

Timeframe	Milestones	Team (p-m)
1 Month	Basic modules: Auth, User, Employee CRUD; Task CRUD & list; DB setup.	2 (dev)
3 Months	Task logs, approvals, basic payroll calc, endpoints + DTOs; initial UI auth (for testing).	4 (dev+test)
6 Months	Full payroll flows, notifications, file uploads, RBAC, multi-tenancy setup.	4 (dev+ops)
12 Months	Advanced: Recurring tasks, integrations (SMS, WhatsApp, calendar), analytics, scaling optimizations.	6 (dev+ops+QA)

Estimated person-months; team size scaled up for complexity (total ~16 p-m).

20. Sources

Key references include Spring Boot and Postgres documentation, AWS S3 guides, and multi-tenancy best practices ¹. Indian context (labor laws, taxes) should follow official guidelines (unspecified here). Additional details adapted from HRMS vendor docs (e.g. task features ²).

¹ Multi-Tenant Architecture: Database Per Tenant vs Shared Schema — Pros and Cons (2026) - DEV Community

https://dev.to/young_gao/multi-tenant-architecture-database-per-tenant-vs-shared-schema-1n2e

² Task Management Software | Employee Task Management System

<https://sawvyhrms.com/task-management/>